24-10-2026

Agenda:

python

Expectation → problem solving

concept ← zero to 100 X

problem ↔ skills ← solution

Network I/O → libraries → requests
→ httpx
→ aiohttp

↓

manage i/o of data across a network.

URL

↓

digital mailing address

google.com

browser

↓

URL → address ⇄ communication → returns HTML (, CI), image) → website

→ data (JSON, xml) → API endpoint

API

communication → protocol

Http / Https

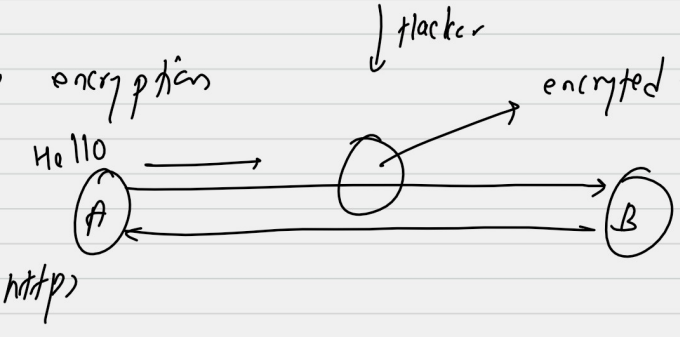
primary purpose	protocol
→ Transferring web pages & api data	Http / Https
→ Real-time, two way communication (like chat app)	websocket
→ Move large files between computers.	FTP / sFTP
→ Send & receive mails	SMTP / Imap
→ Translate address to computer numbers google.com → Ip address	DNS
→ light weight communication for smart home / IOT	MQTT Message Queuing Telemetry Transport
→ video streaming	webRTC

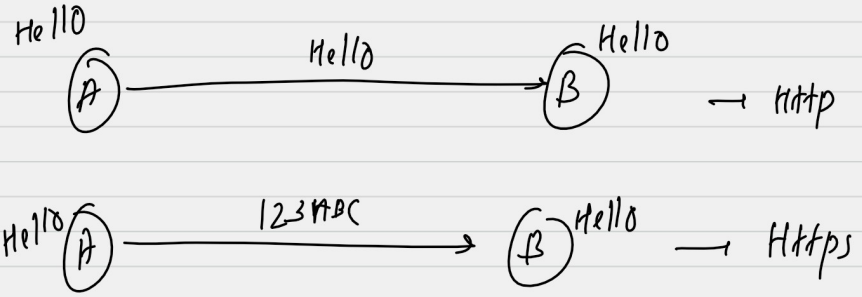
http vs https

http (slow) → data is not secured

2 to 3 trips before sending any data

https → secure (TLS/SSL) → encryption





http://

https://

http → 3

http → 1, 2 → Handshake → SYN → server

network handshake (3 step)

SYN-ACK ←

ACK →

→ encryption (encryption)

↓

https://

http-3 → Handshake/encryption → server

↓

secure

1 round trip

https://

↓

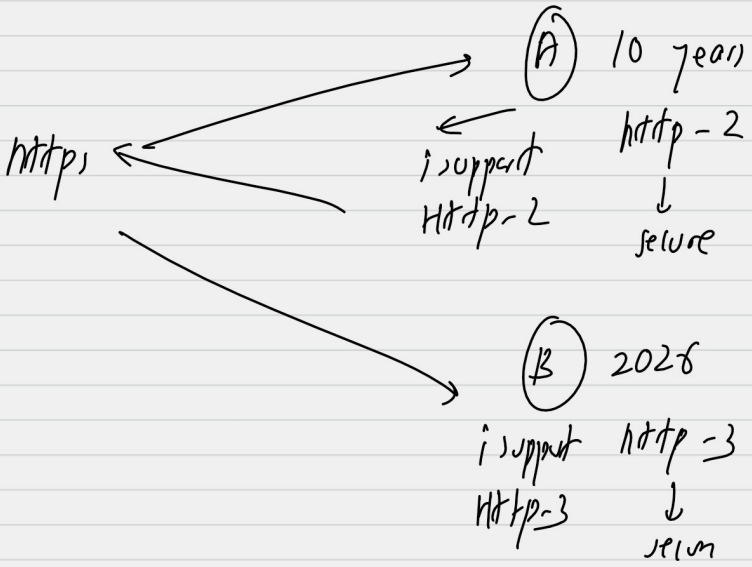
Alt-Svc → h3=:443

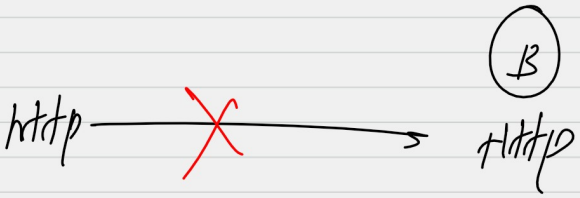
↓

http 3

http3://

https:// → standard





practical

url → data

API

API → retrive data → GET

(no body)

API → retrive data / create a resource

(Body)

{date: 17/10/2026}

→ POST

API → update → PUT

{info}

data

q = {name: krish, batch: all}

replace entirely ← q = {name: kunal, batch: fsdc}

API ⟶ partially ⟶ PATCH

{name: gowri} update

q = {name: krish batch: all}

= {name: gowri, batch: all}

API ⟶ removes q specified resource

(no-body)

DELETE

---

garbage collection

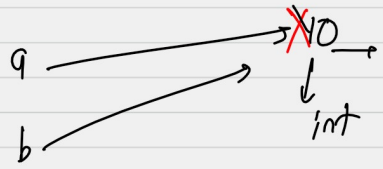
q = 10

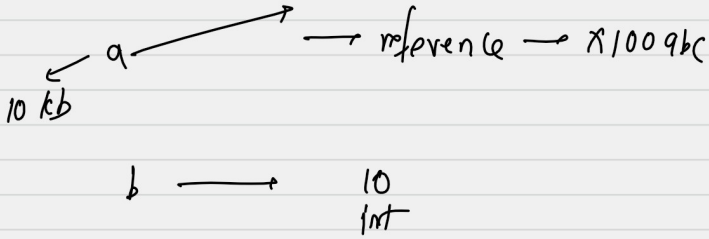
b = q

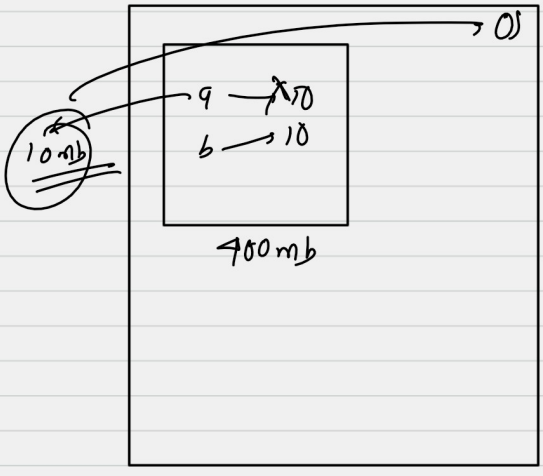
del q

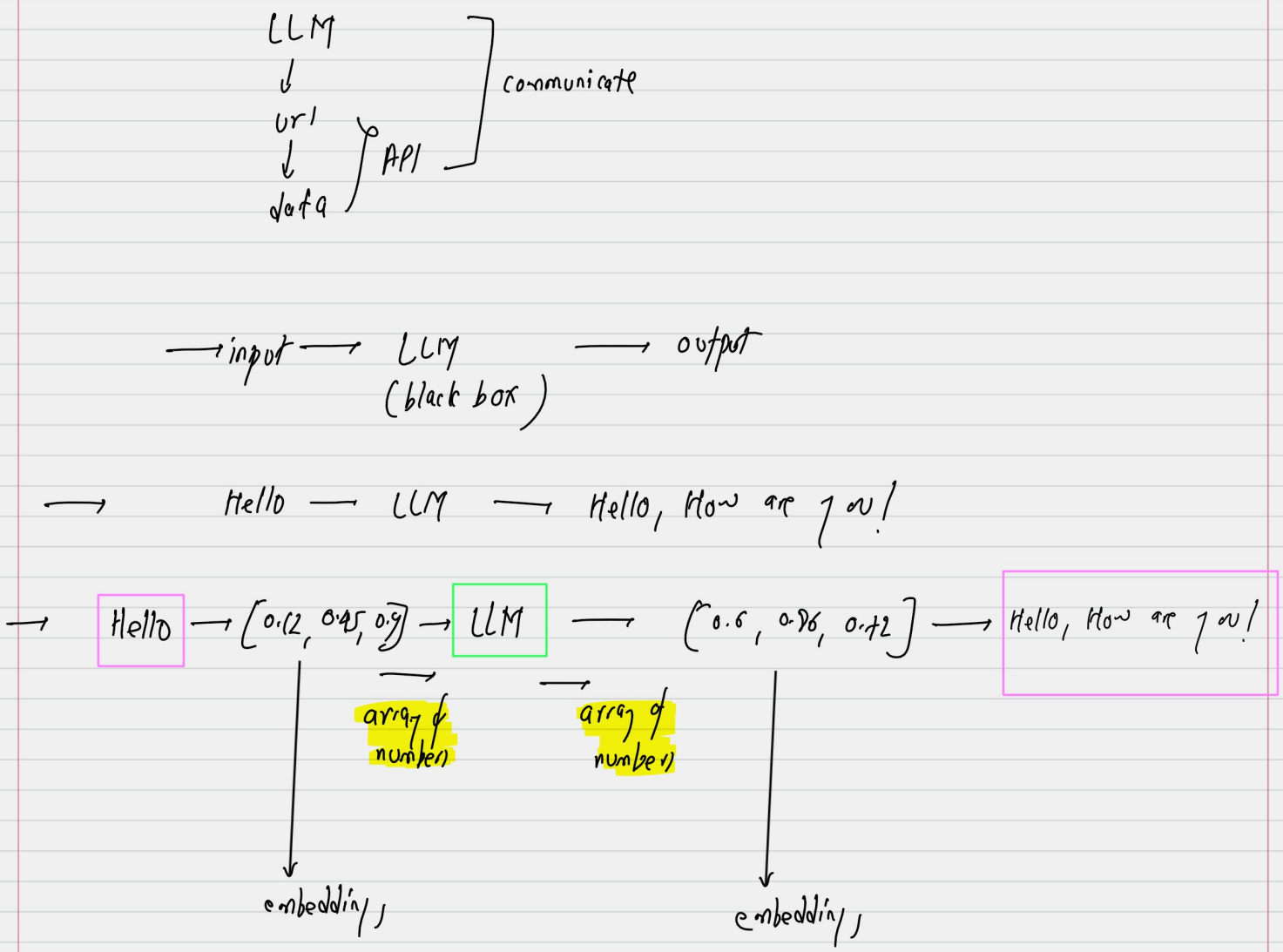
↓

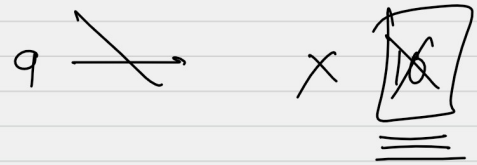
c = 3

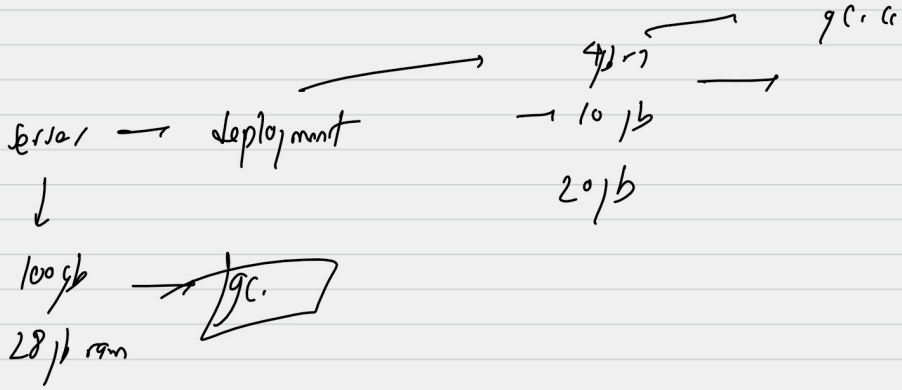







LLM
url
data
API
communicate
input
LLM
(black box)
output
Hello
LLM
Hello, How are you!
Hello
[0.12, 0.45, 0.9]
LLM
[0.6, 0.86, 0.72]
Hello, How are you!
array of numbers
array of numbers
embedding
embedding




server
deployment
10 lb
20 lb
100 gb
28 lb ram
gc.

pdf → software → read_all →

[10,000] → python → data structure → read_all

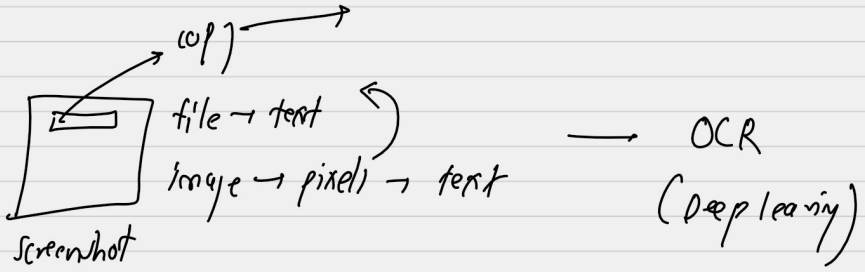
2gb

4gb - 6gb → page by page read

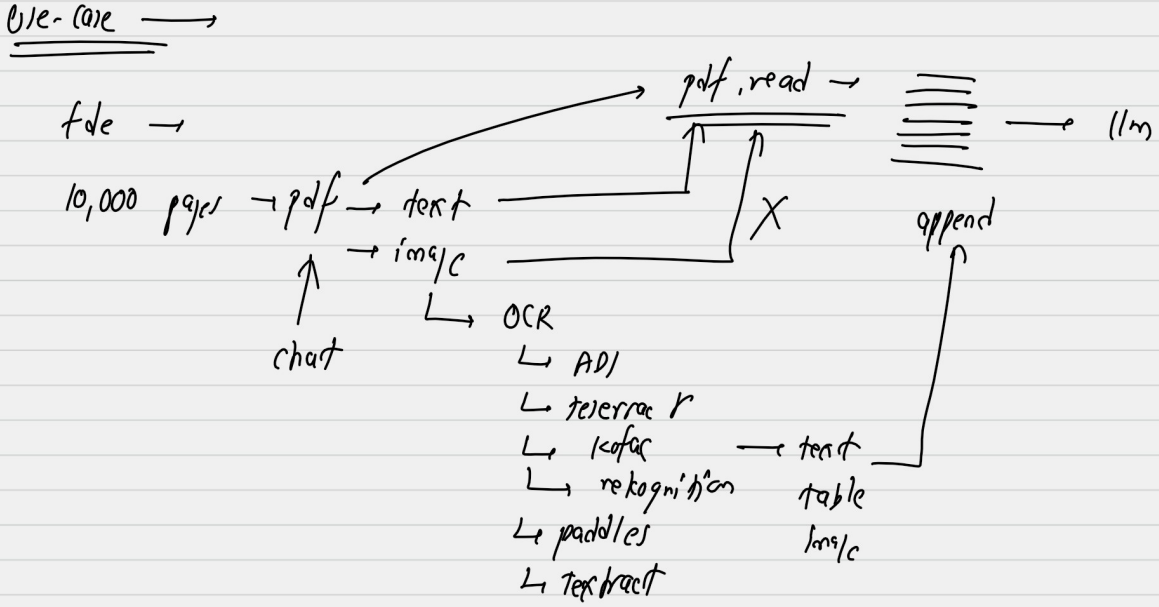
processing

8gb

(llm, api-server)



use-case →



range( ) enumerate( ) → lazy loading function

one by one

generator → yeild

[1, 4, 7, 10, 13, 16, 19, 22, 25]

```
def get_list_values_generator(list_temp):
    for i in list_temp:
        yield i
```

```
q = get_list_values_generator([1, 4, 7, 10, 13, 16, 19, 22, 25])
```

next(q) → 1 → pause(running)

next(q) → 4 → pause(run)

Iterator (class)

→ generator

→ __iter__()

→ __next__()

→ self.__

Automatically manage ✓.

method →

---

request.get(url, param) → api-server → data → 2 gb

1gb json

for i, value in enumerate(full-data):

yield value

response = request.get(url, param, stream=True)

server

os

network queue

1gb

1mb

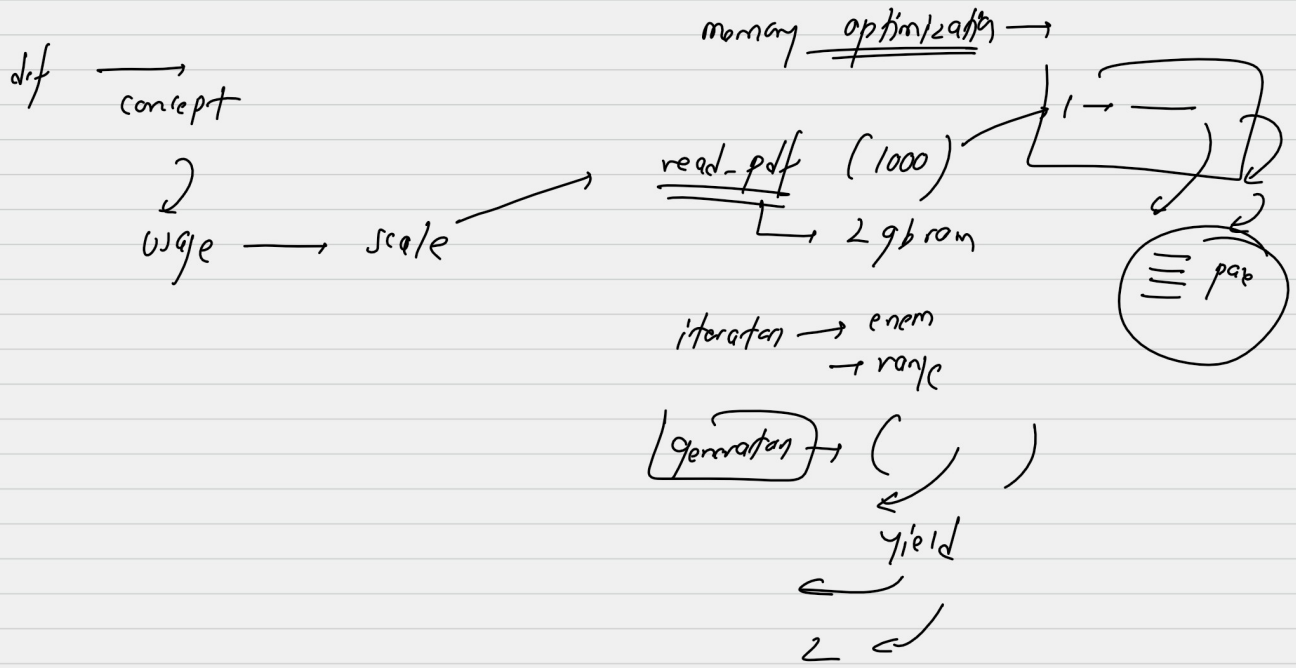
16b

1kb

1g

for i, value in enumerate(response):

yield value


def
concept
usage
scale
memory optimization
read_pdf (1000)
2gb ram
page
iteration
enem
range
generator
yield
2